

Università degli Studi di Salerno

Corso di Ingegneria del Software

**Easy Work Management System
System Design Document
Versione 1.1**



E.W.M.S.

Easy Work Management System

Data: 05/01/2026

Progetto: Easy Work Management System	Versione: 1.1
Documento: System Design Document	Data: 05/01/2026

Coordinatore del progetto:

Nome	Matricola
Giovanni Robustelli	0512121282
Vincenzo Mauro	0512119080

Partecipanti:

Nome	Matricola
Giovanni Robustelli	0512121282
Vincenzo Mauro	0512119080

Scritto da:	Vincenzo Mauro, Giovanni Robustelli
--------------------	-------------------------------------

Revision History

Data	Versione	Descrizione	Autore
25/11/2025	1.0	Assemblaggio documento a partire dai singoli prodotti effettuati dal team	Giovanni Robustelli Vincenzo Mauro
05/01/2026	1.1	Modificati il component e deployment diagram, modificato i servizi offerti dal PersistenceManagement	Giovanni Robustelli

Indice

1.	Introduzione	4
1.1	Scopo del Sistema	4
1.2	Design Goals	4
1.3	Definizioni, Acronimi e Abbreviazioni	8
1.4	Riferimenti	9
1.5	Panoramica del Documento	9
2.	Architettura software corrente	10
3.	Architettura software proposta.....	11
3.1	Overview	11
3.2	Decomposizione in subsystem	12
3.3	Hardware/software mapping	13
3.4	Persistent Data Management.....	14
3.5	Access control and security	15
3.6	Global software control	16
3.6.1	Flusso delle richieste	16
3.6.2	Concorrenza	16
3.6.3	Sincronizzazione e integrità dei dati	16
3.7	Boundary conditions	17
3.7.1	Inizializzazione	17
3.7.2	Terminazione.....	17
3.7.3	Comportamento in caso di errore.....	18
4.	Subsystem Services.....	19
4.1	AccessManagement.....	19
4.2	AccountManagement	19
4.3	TaskManagement	20
4.5	NotificheManagement.....	21
4.6	PersistenceManagement.....	22
	Glossario	23

1. Introduzione

1.1 Scopo del Sistema

E.W.M.S. è una piattaforma di gestione dei task progettata per facilitare la comunicazione tra dipendenti e supervisori in contesti operativi caratterizzati da un elevato volume di attività, come magazzini, centri logistici e reparti industriali.

Il sistema punta a centralizzare l'assegnazione dei compiti e il monitoraggio dello stato delle attività, riducendo i tempi di coordinamento e aumentando l'efficienza e la trasparenza organizzativa.

1.2 Design Goals

DG_1 – UI Web basata su browser moderni

Obiettivo: Garantire che l'interfaccia utente sia completamente fruibile attraverso i principali browser moderni (Chrome, Firefox, Edge, Safari), senza richiedere installazioni locali o software aggiuntivo, massimizzando quanto possibile l'UX, la semplicità di adozione e ridurre i costi di manutenzione relativi ai client.

Trade-off:

- **Compatibilità vs. funzionalità avanzate**
Per garantire la compatibilità cross-browser, è necessario rinunciare o limitare l'uso di funzionalità web ultra-moderne non ancora supportate uniformemente dai browser. Ciò può impedire l'adozione di librerie grafiche o feature molto moderne come API sperimentali o animazioni WebGPU che potrebbero migliorare la UX ma ridurre la compatibilità e le performance.
- **Performance vs. accessibilità**
Garantire l'accesso da browser differenti comporta un design più ottimizzato e standardizzato, che talvolta limita scelte architetturali più performanti o soluzioni proprietarie.

DG_2 – Interfaccia Software Database via JDBC

Obiettivo: Utilizzare JDBC come standard per comunicare con l'RDBMS aziendale assicura portabilità tra diversi DBMS, stabilità, compatibilità con Java e facilità di manutenzione del layer di persistenza.

Trade-off:

- **Portabilità vs. ottimizzazioni specifiche del DBMS**
L'uso di JDBC permette astrarre il DBMS, ma non permette l'implementazione di alcune funzionalità proprietarie che si vorrebbero aggiungere. Ciò limita il massimo livello di performance raggiungibile rispetto a soluzioni più vincolate a un singolo database.
- **Semplicità d'integrazione vs. Complessità delle query**
L'astrazione JDBC richiede spesso la gestione manuale delle query SQL e del mapping dei risultati, aumentando la quantità di codice da scrivere.

DG_3 – Interfaccia WebSocket per notifiche in tempo reale

Obiettivo: Fornire aggiornamenti immediati dell'interfaccia utente tramite WebSocket, permettendo al client di ricevere notifiche push senza refresh della pagina.

Trade-off:

- **Reattività vs. carico sul Server**
Le connessioni WebSocket sono persistenti; quindi, migliorano l'esperienza utente ma aumentano il numero di connessioni aperte sul server, con conseguenti costi di gestione della scalabilità e della memoria.
- **Complessità di manutenzione vs. UX migliorata**
Introduce complessità maggiore rispetto al classico modello request/response. Tuttavia, la UX migliora notevolmente grazie all'aggiornamento immediato del contenuto.

DG_4 – Accesso limitato alla rete intranet

Obiettivo: Garantire che l'applicazione sia accessibile esclusivamente dalla rete intranet aziendale, assicurando un livello di sicurezza più elevato e controllato.

Trade-off:

- **Sicurezza vs. accessibilità esterna**
Limitare l'accesso alla sola intranet riduce enormemente le possibilità di attacco, ma impedisce l'uso esterno (es. supervisor in mobilità, smart working).

Richiede l'eventuale creazione futura di VPN o proxy sicuri se si volesse espandere l'accesso.

- **Affidabilità della rete vs. flessibilità d'uso**

La disponibilità del sistema diventa dipendente in modo critico dall'infrastruttura interna dell'azienda: problemi sulla rete locale significano indisponibilità totale del servizio.

DG_5 – Distribuzione e portabilità del pacchetto

Obiettivo: Garantire che l'applicazione sia facilmente distribuibile come pacchetto **WAR**, compatibile con Apache Tomcat 9+ e JRE 17+.

Tradeoff:

- **Compatibilità vs. ottimizzazione**

La scelta di WAR assicura portabilità e compatibilità standard, ma limita alcune ottimizzazioni specifiche del server o dell'infrastruttura cloud.

- **Stabilità vs. versioni aggiornate**

La dipendenza da versioni specifiche di Tomcat e JRE assicura stabilità, ma può richiedere aggiornamenti frequenti in caso di upgrade dell'ambiente di produzione.

DG_6 – Configurabilità del database

Obiettivo: Consentire la configurazione del database tramite il contesto del server o file di configurazione interni al pacchetto, rendendo l'applicazione flessibile per diversi ambienti.

Tradeoff:

- **Flessibilità vs. complessità**

Esternalizzare la configurazione migliora la portabilità e facilita la gestione di ambienti multipli, ma aumenta la complessità del deploy e la possibilità di errori di configurazione.

- **Semplicità vs. modificabilità**

Configurazioni interne semplificano il deploy, ma rendono più difficile modificare i parametri senza ricreare il pacchetto WAR.

DG_7 – Sicurezza dei dati sensibili

Obiettivo: Proteggere le informazioni personali dei dipendenti tramite cifratura sicura delle password (AES, bcrypt) e accessi profilati per ruolo.

Tradeoff:

- **Sicurezza vs. performance**
Algoritmi di cifratura avanzati garantiscono sicurezza, ma aumentano il carico computazionale durante autenticazione e gestione delle credenziali.
- **Sicurezza vs. manutenibilità**
Controlli di accesso rigorosi riducono il rischio di violazioni, ma complicano lo sviluppo e la manutenzione del sistema.

DG-8 – Gestione sicura delle sessioni

Obiettivo:

Scadenza automatica delle sessioni dopo 30 minuti di inattività, per ridurre il rischio di accessi non autorizzati.

Tradeoff:

- **Sicurezza vs. user experience**
Sessioni più brevi aumentano la sicurezza, ma possono essere frustranti per gli utenti con task prolungati.
Sessioni più lunghe migliorano l'usabilità, ma aumentano la finestra di rischio per accessi non autorizzati.

DG_9 – Prevenzione di attacchi

Obiettivo: Prevenire SQL injection utilizzando **prepared statements** e best practice di validazione input.

Tradeoff:

- **Sicurezza vs. complessità di sviluppo**
L'uso di query parametrizzate aumenta la sicurezza, ma può rendere più complesse query dinamiche o complesse.
La validazione completa dell'input migliora la protezione, ma aumenta il tempo di sviluppo e testing.

1.3 Definizioni, Acronimi e Abbreviazioni

Acronimo	Significato Esteso	Descrizione
ACID	Atomicity, Consistency, Isolation, Durability	Insieme di proprietà che garantiscono la validità delle transazioni nei database.
AES	Advanced Encryption Standard	Algoritmo di cifratura utilizzato per proteggere i dati sensibili.
API	Application Programming Interface	Insieme di definizioni e protocolli per la creazione e l'integrazione di software applicativi.
BLOB	Binary Large Object	Tipo di dato utilizzato nei database per memorizzare oggetti binari di grandi dimensioni (come i file allegati).
CSS	Cascading Style Sheets	Linguaggio usato per definire la formattazione e lo stile delle pagine web.
DBMS	Database Management System	Sistema software progettato per consentire la creazione, la manipolazione e l'interrogazione di database.
ER	Entity-Relationship	Modello per la rappresentazione concettuale dei dati e delle loro relazioni.
HTML	HyperText Markup Language	Linguaggio di markup standard per la creazione di pagine web.
HTTP / HTTPS	HyperText Transfer Protocol (Secure)	Protocollo di rete a livello applicativo utilizzato per la trasmissione di informazioni sul web (la versione Secure usa crittografia SSL/TLS).
JDBC	Java Database Connectivity	API standard del linguaggio Java che definisce come un client può accedere a un database.
JRE	Java Runtime Environment	Ambiente di esecuzione necessario per eseguire applicazioni scritte in Java.
JSP	JavaServer Pages	Tecnologia Java che consente agli sviluppatori di creare pagine Web dinamiche basate su HTML, XML o altri tipi di documenti.
JVM	Java Virtual Machine	Macchina virtuale che esegue i bytecode Java.
MIME	Multipurpose Internet Mail Extensions	Standard che estende il formato delle e-mail e viene usato nel web per indicare il tipo di contenuto di un file (es. application/pdf).

SQL	Structured Query Language	Linguaggio standardizzato per database basati sul modello relazionale.
TCP/IP	Transmission Control Protocol / Internet Protocol	Insieme di protocolli di comunicazione usati per interconnettere dispositivi di rete su internet.
WAR	Web Application Archive	Formato di file JAR utilizzato per distribuire una collezione di file JSP, servlet, classi Java e risorse web.
UI	User Interface	Interfaccia Utente, il mezzo attraverso cui l'utente interagisce con il sistema.
UX	User Experience	Riguarda le percezioni e le risposte di una persona risultanti dall'uso del sistema.

1.4 Riferimenti

I seguenti documenti sono stati usati come riferimento per la stesura di questo documento di analisi dei requisiti:

[REF-1] Object-oriented-Software-Engineering-3rd-Edition

[REF-2] RAD_EWMS V1.0

[REF-3] Team di Progetto, SDD_start_meeting

1.5 Panoramica del Documento

Il documento fornisce una descrizione dettagliata dell'architettura software progettata per il sistema Easy Work Management System (E.W.M.S.).

Il documento è organizzato nelle seguenti sezioni:

- **Sezione 2 – Architettura software corrente:** Descrive la situazione attuale presso il cliente. Poiché non esiste un sistema software precedente, questa sezione analizza i flussi di lavoro manuali attuali e ne evidenzia le criticità che il nuovo sistema intende risolvere.
- **Sezione 3 – Architettura software proposta:** Costituisce il nucleo del design e definisce l'architettura tecnica del nuovo sistema E.W.M.S.
 - Viene presentata la scomposizione logica in sottosistemi.
 - Viene definita la mappatura tra componenti software e nodi hardware.

- Vengono dettagliate le strategie per la persistenza dei dati, la sicurezza e la gestione del flusso di controllo globale.
- Vengono descritte le condizioni di confine.
- **Sezione 4 – Subsystem services:** Definisce le interfacce e i servizi pubblici offerti da ciascun sottosistema, specificando le operazioni disponibili per gli attori (Dipendente, Supervisore, Gestore) e per gli altri moduli del sistema.
- **Glossario:** Fornisce le definizioni dei termini tecnici e degli acronimi utilizzati nel documento.

2. Architettura software corrente

Questa parte del documento analizza la struttura delle architetture correnti, ne sottolinea le inefficienze e le limitazioni.

Confrontando queste modalità con il nostro sistema, si può notare come E.W.M.S. offra flessibilità, adattabilità e usabilità nella gestione dei task, rendendo più facile il lavoro sia per i dipendenti che per i supervisori.

Molte aziende fanno ancora affidamento su strumenti manuali o semi-digitali, come fogli di calcolo condivisi, task list cartacee, comunicazioni tramite telefono o messaggistica istantanea, oppure moduli generici inclusi in suite aziendali non specializzate: questi approcci non offrono scalabilità, controllo centralizzato né visibilità in tempo reale.

La progettazione del sistema parte dall'assunzione che siano necessari:

- accesso immediato alle informazioni operative
- aggiornamenti dello stato in tempo reale
- comunicazioni strutturate e trasparenza nelle attività.

Gli strumenti attuali risultano insufficienti a supportare la complessità e le esigenze di tracciabilità tipiche degli ambienti molto dinamici ed esigenti. I problemi più comuni riscontrati includono: frammentazione delle informazioni, mancanza di un controllo centralizzato dei task, impossibilità di monitorare efficacemente l'avanzamento, carico comunicativo elevato su canali non strutturati, scarsa scalabilità, bassa integrazione con i flussi operativi reali, ridondanza delle informazioni e perdita di dati. E.W.M.S. affronta tali limitazioni offrendo una piattaforma centralizzata, progettata per garantire visibilità in tempo reale, scalabilità operativa e un'interazione fluida tra dipendenti e supervisori. L'architettura del sistema permette una gestione precisa dei task e una distribuzione efficiente del carico operativo.

3. Architettura software proposta

3.1 Overview

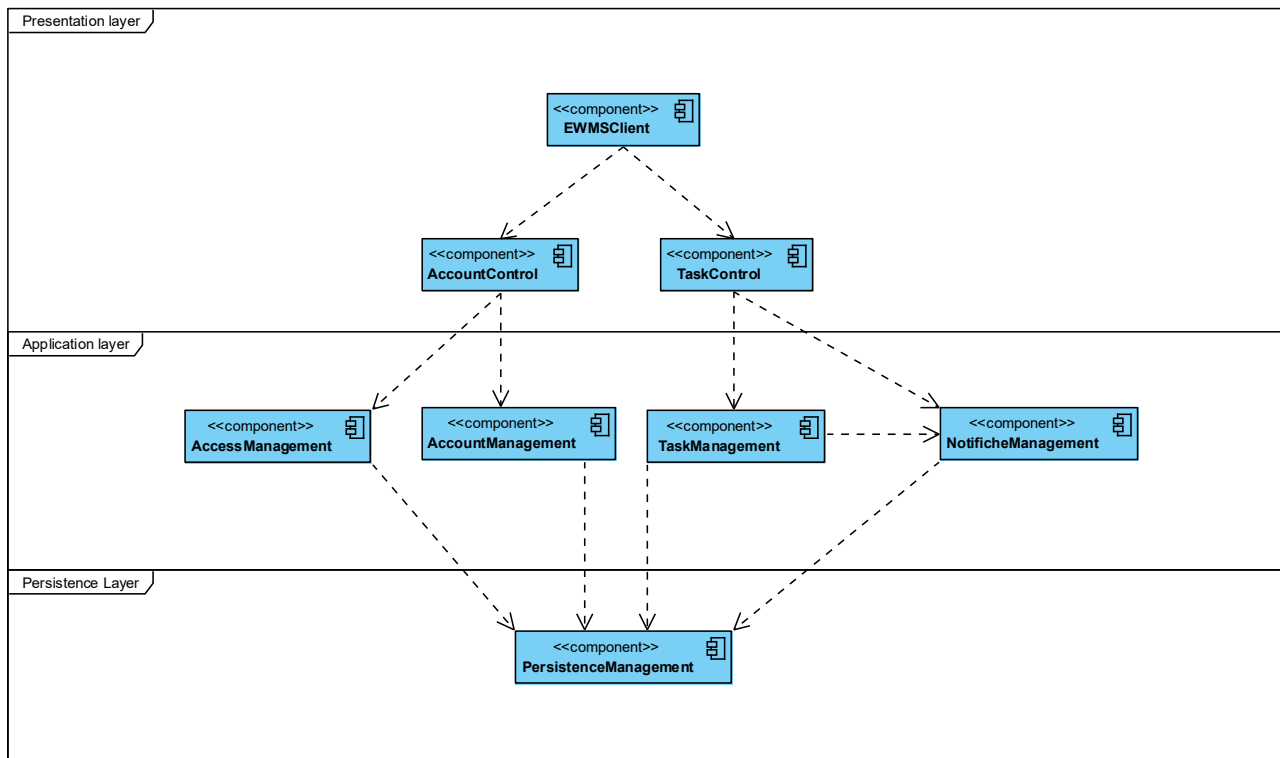
Il sistema Easy Work Management System (E.W.M.S.) è progettato seguendo un'architettura software Three-Tier (a tre livelli) basata sul web. L'architettura segue il design pattern Model-View-Controller (MVC) per l'organizzazione della logica applicativa.

Il sistema è strutturato quindi su tre livelli:

- **Presentation layer:** costituisce il livello di presentazione accessibile agli utenti finali. L'interfaccia è fruibile attraverso qualsiasi browser web moderno. Questo livello è responsabile della visualizzazione delle pagine dinamiche (generate tramite tecnologia) e dell'invio delle richieste utente al server tramite protocollo HTTP/HTTPS.
- **Application layer:** costituisce il cuore del sistema, ospitato su un Web Server Apache Tomcat. Questo livello implementa la logica di business e il controllo del flusso applicativo.
- **Persistence layer:** Costituisce il livello di persistenza dei dati. È composto da un Database Management System (DBMS) relazionale, il server applicativo comunica con questo livello utilizzando lo standard **JDBC**.

3.2 Decomposizione in subsystem

Di seguito viene presentato il sistema scomposto in “subsystem” e le relative dipendenze.

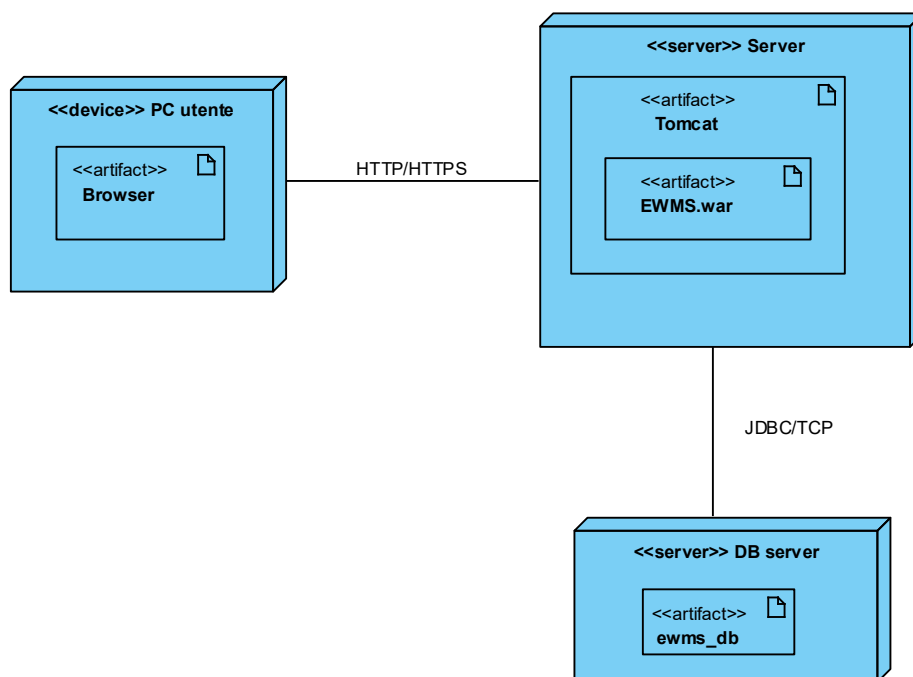


Nome	Descrizione
EWMSClient	Sottosistema che si occupa della presentazione lato client offre tutti i servizi che verranno utilizzati dagli utenti finali per comunicare con il sistema
AccountControl	Sottosistema che si occupa delle operazioni di controllo di tutto quello che riguarda gli account.
TaskControl	Sottosistema che si occupa delle operazioni di controllo di tutto quello che riguarda i task
AccessManagement	Sottosistema che offre i servizi per login e logout dal proprio account
AccountManagement	Sottosistema che si occupa dei servizi di modifica e inserimento degli account sul database del sistema
TaskManagement	Sottosistema che offre tutti i servizi che si concentrano sul controllo del ciclo di vita di un task all'interno del sistema
NotificheManagement	Sottosistema che offre i servizi di controllo delle notifiche generate dalle operazioni sui task
PersistenceManagement	Sottosistema che offre i servizi per la lettura e scrittura dal database

3.3 Hardware/software mapping

Il sistema E.W.M.S. è distribuito su tre nodi computazionali distinti, corrispondenti ai livelli dell'architettura Three-Tier:

- **Client Node:** il dispositivo dell'utente finale, essendo EWMS una web application che gira sul server di azienda non richiede installazione di software proprietario, ma si affida ad un browser (Chrome, Mozilla, Opera ecc...).
- **Application Server Node:** È il server centrale che ospita la logica applicativa. Esegue il container web necessario per gestire il ciclo di vita delle Servlet e delle pagine JSP.
- **Database Server Node:** È il server dedicato alla persistenza dei dati. Ospita il DBMS relazionale che memorizza tutte le informazioni di sistema.



Componenti off-the-shelfs

Componente	Tipo	Descrizione
Apache Tomcat	Web Server/Container	Gestione del ciclo di vita di Servlet/JSP e dove verrà caricato il file ewms.war per il deployment
MySQL	DBMS	Gestione della persistenza dei dati
Java SE JVM	Runtime enviroment	Ambiente di esecuzione per il codice Java lato server
WebBrowser	Client runtime	Gestione del rendering delle pagine web generate dal server, supporta HTML5 e CSS3

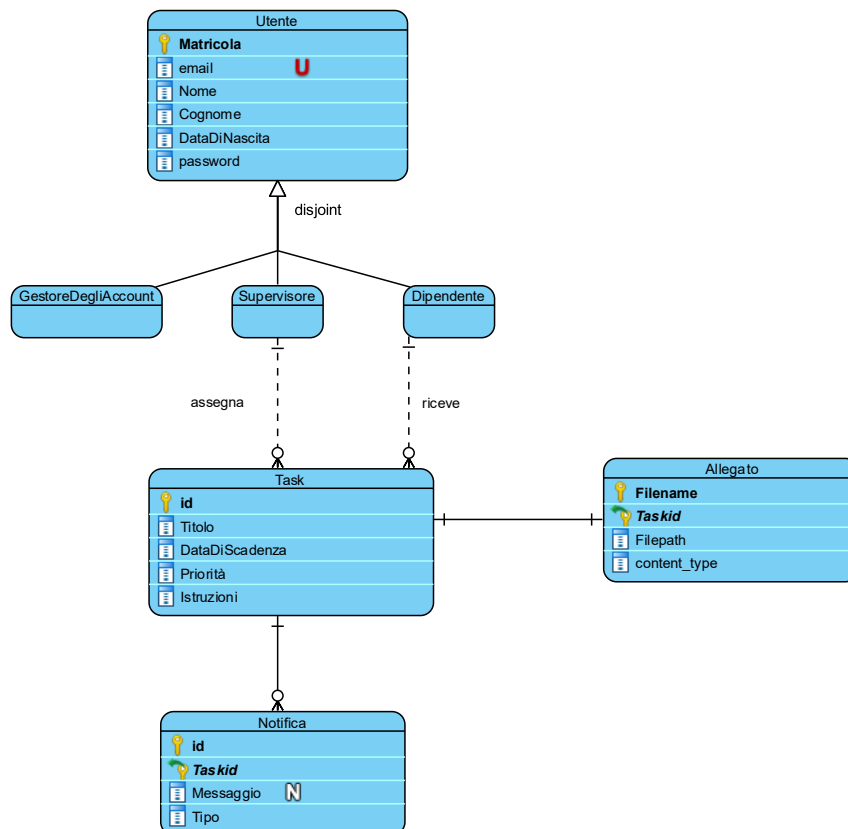
3.4 Persistent Data Management

La gestione della persistenza dei dati nel sistema E.W.M.S. è affidata a un RDBMS (Relational Database Management System). Il sistema applicativo interagisce con il database per garantire che le informazioni relative agli account, ai task e alle notifiche siano salvate in modo durevole, consistente e sicuro.

Il DBMS utilizzato sarà MySQL questo ci permetterà di soddisfare le esigenze di **compatibilità** essendo presente un supporto nativo e driver stabili per lo standard JDBC, utilizzato dal livello applicativo. Per ottimizzare le prestazioni del database, i file binari (allegati) non verranno memorizzati direttamente come BLOB nel RDBMS. Il sistema utilizzerà un approccio su due livelli:

- I **file fisici** verranno salvati in una directory del server applicativo.
- La tabella “Allegato” memorizzerà solo i metadati necessari al recupero del file:
 - o Filename: il nome del file caricato,
 - o File_path: dove trovare lato server il file caricato,
 - o Content_type: il tipo MIME (es. pdf, .xlsx ecc...)

Schema ER



3.5 Access control and security

La strategia di sicurezza del sistema E.W.M.S. è progettata per garantire la riservatezza e l'integrità dei dati attraverso un modello di controllo degli accessi basato sui ruoli (Role-Based Access Control). L'implementazione del controllo degli accessi verrà effettuata a livello programmatico tramite l'utilizzo di **Servlet Filters** standard. Questi componenti intercettano ogni richiesta HTTP diretta al livello applicativo, verificando l'identità dell'utente e i suoi privilegi prima di consentire l'esecuzione di qualsiasi logica di business. Viene definita di seguito la Matrice di Accesso che specifica quali operazioni un attore può effettuare su un oggetto.

<i>Attore</i> \ Oggetto	Utente	Notifica	Task	Allegato
Supervisore	login change_pwd	showNotification sendWarning holdTask	createTask deleteTask showTasks fetchTask holdTask	upload download
Dipendente	Login change_pwd	showNotification holdTask	showTasks fetchTask inizializzaTask completeTask holdTask	download
Gestore degli account	Login change_pwd addAccount getAccount generateNewPwd modifyRole modifySupervisor deleteAccount			

3.6 Global software control

Il controllo del flusso software nel sistema E.W.M.S. segue il modello Event-Driven, tipico delle applicazioni web interattive. L'esecuzione è innescata dalle azioni degli utenti (richieste HTTP) e gestita dal Web Container (Apache Tomcat).

3.6.1 Flusso delle richieste

Il flusso di controllo principale è del tipo sincrono Request-Response:

1. L'evento scatenante è una richiesta http (GET o POST) originata dal browser lato client che interagisce con la pagina;
2. Il web-container intercetterà la richiesta, istanzierà un nuovo thread dedicato e lo indirizza alla servlet specificata;
3. La servlet esegue la logica di business;
4. Al termine il controllo passa a una pagina JSP che genera l'HTML di risposta inviato al client.

3.6.2 Concorrenza

Essendo EWMS un applicativo web, la concorrenza sarà gestita dal Web-container. Tomcat mantiene un Thread Pool; ogni richiesta HTTP verrà quindi eseguita in un thread separato.

Le servlet, inoltre, sono oggetti di tipo stateless, non conservano uno stato e non utilizzano variabili di istanza mutabili per conservare dati specifici dell'utente, prevenendo **race condition** quando più thread accedono alla stessa Servlet contemporaneamente.

Lo stato dell'utente verrà gestito grazie all'utilizzo di un oggetto HttpSession unico per ogni utente garantendo così l'isolamento dei dati.

3.6.3 Sincronizzazione e integrità dei dati

La sincronizzazione tra i sottosistemi e l'accesso concorrente al database è gestito attraverso:

- **Transazioni Database (ACID):** Per operazioni critiche che coinvolgono più scritture, la consistenza è delegata al DBMS tramite transazioni. In caso di errore in uno dei passaggi, l'intera operazione viene annullata (Rollback).
- **Connection Pooling:** L'accesso al database è sincronizzato tramite un DataSource (Connection Pool). Questo componente gestisce una coda di connessioni disponibili, mettendo in attesa i thread se tutte le connessioni sono occupate, prevenendo il sovraccarico del database.

- **Chiamate tra Sottosistemi:** La comunicazione tra i moduli logici avviene tramite invocazione sincrona di metodi Java.

3.7 Boundary conditions

In questa sezione viene descritto il comportamento del sistema E.W.M.S. durante le fasi di avvio, arresto e in presenza di errori critici. L'obiettivo è garantire l'integrità dei dati e fornire feedback adeguati agli utenti e ai programmatori che interverranno in caso di malfunzionamenti.

3.7.1 Inizializzazione

L'avvio del sistema coincide con il deploy del file EWMS.war all'interno del Web Container (Apache Tomcat) o con il riavvio del servizio Tomcat stesso. Durante la fase di inizializzazione, il sistema esegue le seguenti operazioni sequenziali:

1. **Caricamento Configurazione:** Il sistema legge i parametri di configurazione dal file web.xml e dalle variabili d'ambiente del server.
2. **Inizializzazione Connection Pool:** Viene istanziato il DataSource per la connessione al database MySQL.
 - *Eccezione:* Se il database non è raggiungibile, l'applicazione non completa l'avvio e logga un errore critico (FATAL), impedendo l'accesso agli utenti.
3. **Verifica File System:** Il sistema controlla l'esistenza e i permessi di scrittura della directory dedicata agli allegati (definita nella sezione 3.4). Se la cartella non esiste, il sistema tenta di crearla.
4. **Ready State:** Una volta completati questi passaggi, l'applicazione inizia ad accettare le richieste HTTP in ingresso.

3.7.2 Terminazione

La terminazione del sistema coincide all'arresto o l'undeploy, tramite il servizio Tomcat. Il sistema garantisce:

1. **Chiusura Sessioni:** Le sessioni utente attive vengono invalidate. I dati di sessione non persistenti vengono persi, mentre lo stato dei task su database rimane consistente.
2. **Rilascio Risorse Database:** Il Connection Pool viene chiuso forzatamente. Tutte le connessioni JDBC attive vengono terminate e i driver database deregistrati per evitare *memory leaks* nel server.

3. **Completamento Thread:** Se vi sono operazioni di upload file in corso, il sistema tenta di completarle (entro un timeout prestabilito) o di abortirle in modo pulito cancellando i file parziali.

3.7.3 Comportamento in caso di errore

Il sistema deve gestire i malfunzionamenti garantendo che l'utente non veda mai stack trace o errori di codice.

A. Errori Applicativi (Software Bugs / Eccezioni) In caso di NullPointerException o altre eccezioni non gestite (HTTP 500):

- **Log:** L'errore completo (con stack trace) viene scritto nei file di log del server (es. Log4j/SLF4J) per l'analisi da parte degli sviluppatori.
- **User Experience:** L'utente viene reindirizzato automaticamente a una pagina di errore (error500.jsp) che comunica "Si è verificato un errore imprevisto" e invita a riprovare più tardi, senza mostrare dettagli tecnici sensibili.

B. Indisponibilità del Database Se il database MySQL diventa irraggiungibile durante l'operatività:

- Il Connection Pool solleverà eccezioni SQLException.
- Il sistema intercetta queste eccezioni e mostra una pagina specifica di "Servizio non disponibile momentaneamente", impedendo ulteriori tentativi di scrittura che potrebbero portare a inconsistenza.

C. Errori di Navigazione (HTTP 404) Se l'utente tenta di accedere a una risorsa inesistente o a un URL malformato:

- Il sistema mostra una pagina personalizzata (error404.jsp) con un link per tornare alla Dashboard principale, mantenendo l'utente all'interno dell'esperienza applicativa.

D. Violazioni di Sicurezza Come definito nella sezione 3.5, tentativi di accesso non autorizzato (es. URL Forcing) non generano errori di sistema, ma reindirizzano l'utente alla pagina di Login o mostrano una pagina di errore 403 (Forbidden) standardizzata.

4. Subsystem Services

In questa sezione verranno descritti i servizi offerti da ciascun subsystem applicativo.

4.1 AccessManagement

SessionService: gestisce il controllo degli accessi e la sicurezza della sessione. È utilizzato da tutti gli altri sistemi e dal livello client per verificare l'identità e i permessi.

Operazione	Descrizione
login	Verifica le credenziali dell'utente e crea una nuova sessione.
Logout	Invalida la sessione corrente dell'utente.
checkPermission	Verifica se l'utente corrente ha i permessi per eseguire un'azione specifica.

4.2 AccountManagement

PersonalProfileService: gestisce tutte le operazioni che un utente autenticato può eseguire sul proprio account

Operazione	Descrizione
getMyProfile	Recupera i dettagli dell'account dell'utente correntemente loggato.
changePwd	Permette all'utente di aggiornare la propria password.

ProfileManagementService: gestisce le operazioni amministrative riservate al ruolo del Gestore degli account

Operazione	Descrizione
addAccount	Permette di aggiungere un nuovo account al sistema.
getAccount	Permette di recuperare i dati di un account selezionato dal client.
generateNewPwd	Permette di rimpiazzare la password preesistente di un account con una nuova.
modifyRole	Permette di cambiare il ruolo assegnato ad un account.
modifySupervisor	Permette di cambiare supervisore ad un dipendente.
deleteAccount	Permette di eliminare un account dal sistema.

4.3 TaskManagement

TaskCommonService: espone le operazioni di lettura e le azioni condivise accessibili sia ai Dipendenti che ai Supervisor.

Operazione	Descrizione
getTask	Recupera le informazioni complete di un task.
filterTask	Permette di filtrare task per stato (da completare, inizializzate ecc...).
holdTask	Permette di cambiare sospendere un task e cambiare di conseguenza lo stato.
dowloadAllegato	Permette di scaricare un allegato dal task in cui è stato caricato.

TaskSupervisoreService: espone tutte le operazioni eseguibili da un Supervisore sul ciclo di vita di un task

Operazione	Descrizione
createTask	Permette di assegnare un task ad un dipendente.
deleteTask	Permette di rimuovere un task precedentemente assegnato dal sistema e notifica il dipendente.
sendWarning	Invia un sollecito o un avviso formale al dipendente assegnato al task.

TaskDipendenteService: espone tutte le operazioni eseguibili da un Dipendente sul ciclo di vita di un task.

Operazione	Descrizione
inizializzaTask	Segnala l'inizio dei lavori da parte di un dipendente, portando il task dallo stato "Da completare" a "In elaborazione".
completeTask	Segnala la conclusione del lavoro, portando il task nello stato finale "Completato".

4.5 NotificheManagement

NotificheService: gestisce la comunicazione asincrona e gli avvisi.

Operazione	Descrizione
showNotification	Recupera le notifiche per l'utente corrente
send	Permette di inviare notifiche verso un determinato ricevente.

4.6 PersistenceManagement

PersistenceService: gestisce tutte le operazioni per rendere i dati persistenti nel database e nella cartella degli allegati.

Operazione	Descrizione
getUtenteDAO	Restituisce un riferimento ad un oggetto della classe UtenteDAO
getTaskDAO	Restituisce un riferimento ad un oggetto della classe TaskDAO
getNotificheDAO	Restituisce un riferimento ad un oggetto della classe NotificheDAO

Glossario

Web Container: L'ambiente software (nel nostro caso Apache Tomcat) che "ospita" l'applicazione. Fornisce tutto ciò che serve al codice Java per funzionare, gestire le richieste web e parlare con il sistema operativo.

Deploy / Deployment: L'atto di installare e avviare l'applicazione sul server di produzione. È il momento in cui il codice scritto dai programmatori viene impacchettato e messo in condizione di essere usato dagli utenti.

Graceful Shutdown: Una procedura di spegnimento controllato.

Connection Pool: Una tecnica per migliorare le prestazioni. Invece di aprire e chiudere una connessione al database per ogni singola richiesta (operazione lenta), il sistema mantiene un pool (insieme) di connessioni sempre aperte e pronte all'uso, prestandole ai vari processi quando ne hanno bisogno.